

Minecraft for Semantic Segmentation: A Procedural Data Pipeline and Sim-to-Real Analysis

Duong Vu

Massachusetts Institute of Technology
Cambridge, Massachusetts, USA

duongvu@mit.edu

Abstract

Synthetic data has emerged as a partial solution to the high cost of pixel-level annotation for semantic segmentation, with prior work primarily drawing on photorealistic 3D environments. We investigate the opposite end of the fidelity spectrum: can a deliberately low-fidelity, voxel-based source produce useful training data? We present a fully automated pipeline that converts OpenStreetMap data into Minecraft worlds (via Arnis), extracts pixel-perfect semantic labels through a custom flat-color resource pack, and produces integer-valued masks via lightweight post-processing, with no manual annotation and no modifications to the Minecraft engine. Fine-tuning SegFormer-B2 on 300 paired frames of central Frankfurt yields 0.91 mIoU in-distribution but only 0.45 on Cityscapes, substantially below the same model evaluated zero-shot from its ADE20K-pretrained weights, which yields 0.90. A per-class analysis reveals a structured gap: large position-defined regions (sky, dominant built structure) transfer measurably, while object categories absent from the synthetic source (vehicles, pedestrians, urban furniture) drive most of the failure through default-class substitution. Code and data-generation pipeline are available at <https://github.com/duongvu2005/minecraft-sim2real>.

1. Introduction

Scene segmentation, the task of assigning a class label to every pixel in an image, is a foundational problem in computer vision and a building block for autonomous driving, robotic perception, and scene understanding. Modern models achieve strong performance on this task, but their accuracy is bottlenecked by the availability of large, pixel-accurately labeled datasets.

Producing such datasets with manual labeling is uniquely expensive. Real-world images are inherently ambiguous, as object boundaries are rarely sharp and small or

distant objects can be challenging to identify. For instance, Cityscapes—a widely used benchmark dataset of urban street scenes for semantic segmentation—requires approximately 90 minutes of human labor to annotate a single image [3]. The result is a scaling ceiling: the more detailed the labeling task, the smaller the dataset that can be produced. This is a critical limitation given that state-of-the-art deep learning models appear to be limited by data rather than model capacity.

To address this bottleneck, Richter *et al.* [12] proposed automatically extracting pixel-accurate semantic labels from commercial video games, bypassing human annotation almost entirely. They demonstrated this approach using the game Grand Theft Auto V (GTA5), producing a dataset of approximately 25,000 densely annotated frames in just 49 hours. This averages to 7 seconds per image, which is 771 times faster than the manual annotation time for the Cityscapes dataset. Crucially, combining this synthetic data with real-world images during training significantly improves segmentation performance: a model trained on GTA5 data alongside just one-third of the CamVid training set [2] outperformed a model trained on the full CamVid set. This demonstrates that game engines can serve as a viable source of labels. Synthetic data can also generate rare scenarios and sidestep privacy concerns inherent to recording real people and places.

Underlying this success is a shared assumption: the synthetic source is photorealistic, or at least highly realistic in terms of geometry, lighting, and textures. In the example above, GTA5 was chosen precisely for its photographic fidelity, implicitly treating visual realism as a prerequisite for useful synthetic data. Whether fidelity is itself necessary for useful synthetic data (and how transfer scales as fidelity drops) remains open, with practical stakes: low-fidelity environments are dramatically cheaper to set up, modify, and scale.

We investigate this question at the far low-fidelity end. Minecraft, a video game powered by a voxel-based engine with blocky geometry and cartoonish tiled textures,

is a natural proving ground; it already has an established role as a training environment for reinforcement learning agents [1, 6, 9, 10]. We invert that perspective and ask whether Minecraft itself can serve as a *source* of training data for vision models.

Answering this requires obtaining labeled segmentation data from Minecraft worlds at scale, in a form that resembles real urban scenes. Default Minecraft worlds are procedurally generated natural landscapes, so scene composition does not come for free, unlike prior work using games like GTA5. We therefore design a pipeline that combines three components: Arnis [5] converts OpenStreetMap data [11] into a voxelized urban world; a flat-color resource pack assigns each block a class-encoded color to yield pixel-perfect masks; and a lightweight post-processing step converts those colors to integer labels.

We use this pipeline to generate 300 paired RGB-mask frames from a Minecraft rendering of central Frankfurt and fine-tune SegFormer-B2 [17] on a four-class scheme (road, structure, vegetation, sky). We evaluate the model on held-out Minecraft frames and the Cityscapes [3] validation set, and compare against an ADE20K-pretrained [18] SegFormer applied zero-shot. The model achieves 0.91 mIoU in-distribution but only 0.45 on Cityscapes, substantially below the baseline’s 0.90. Per-class analysis reveals a structured pattern: large position-defined regions (sky, dominant structures) transfer measurably, while object categories absent from Minecraft induce systematic over-prediction of the structure-class, degrading the surrounding classes.

Our main contributions are:

- A pipeline for low-cost, fully automated semantic segmentation data generation from procedural Minecraft worlds, combining OpenStreetMap-derived geometry (Arnis), a flat-color resource pack for label extraction, and lightweight post-processing with no manual annotation and no engine modification.
- An empirical characterization of low-fidelity sim-to-real transfer, comparing a Minecraft-trained SegFormer model against a strong ADE20K-pretrained baseline on Cityscapes with detailed per-class IoU and confusion-matrix analysis.
- A characterization of failure modes specific to low-fidelity synthetic sources, identifying which categories transfer (large position-defined regions) and which do not (object categories absent from the synthetic source).
- Open-source release of the pipeline and trained models.

2. Related Work

Synthetic data for semantic segmentation. Beyond Richter *et al.*, several other synthetic datasets have been used to train urban-scene segmentation models. SYNTHIA [13] and Virtual KITTI [7] both provide photorealistic 3D environments rendered with pixel-level annotations

for urban scene understanding. A complementary line abandons photorealism: domain randomization, proposed by Tobin *et al.* [14] and extended to driving scenes [15], generates deliberately non-realistic textures, encouraging models to rely on shape and layout rather than texture. Common to all of these is that the synthetic source is tailored to the target vision task: SYNTHIA and Virtual KITTI render urban driving scenes specifically because they target driving benchmarks; Tobin *et al.* construct a tabletop simulator that mirrors the robot’s workspace. Our work challenges this assumption: Minecraft is a general-purpose game with no target-vision task, and our pipeline imposes task-relevant geometry on it post hoc.

Domain adaptation and the sim-to-real gap. A large body of work closes the sim-to-real gap through domain adaptation rather than by improving the synthetic source. Domain stylization [4] applies neural style transfer to make synthetic images visually resemble target-domain photographs before training. Adversarial approaches built on AdaptSegNet [16] align feature distributions between synthetic and real domains during training. We deliberately study the unadapted regime to isolate what transfers from low-fidelity synthetic data on its own; adaptation methods are orthogonal and could be combined with our pipeline in future work.

Architectural and theoretical context. Two further lines inform our experimental design. Geirhos *et al.* [8] showed that ImageNet-trained convolutional networks are biased toward texture rather than shape, and that increasing shape bias improves robustness under distribution shift. This is directly relevant to our setting: Minecraft preserves the *shape* and *layout* of urban scenes (via Arnis) while drastically altering *texture*, so a shape-biased model should suffer less from this kind of domain shift. We use SegFormer [17], a transformer architecture argued to be more shape-biased than its CNN counterparts, initialized with weights pretrained on ADE20K [18].

3. Methodology

3.1. Pipeline overview

Our pipeline produces paired RGB-mask data in three stages.

Stage 1: World generation. Arnis converts OpenStreetMap data into a Minecraft world whose geometry mirrors a real city.

Stage 2: Paired capture. At each viewpoint, we capture two screenshots: one with the default texture pack (RGB), and one with a custom flat-color resource pack (mask), with viewpoint and lighting held fixed between the two.

Stage 3: Label post-processing. Raw color masks are converted to integer class maps via a per-pixel classifier that accounts for per-frame lighting variation and

the residual sky gradient. The output is a directory of paired (`rgb_NNNN.png`, `mask_NNNN.png`) files, drop-in compatible with any existing semantic segmentation pipeline. We use a four-class scheme, namely road, structure, vegetation, and sky, chosen so that every visible Minecraft block can be unambiguously assigned a class and the result maps coherently onto Cityscapes labels. The structure class aggregates buildings, walls, fences, and side-walks because the Minecraft block taxonomy does not consistently distinguish among them. We collected 300 paired frames from a single Frankfurt-derived world; collection was manual in this work, but is automatable in principle (see [section 6](#)).

3.2. World generation with Arnis

We use Arnis [5], an open-source tool that maps OSM features such as buildings, roads, and vegetation to Minecraft blocks, to export a bounding box centered on Frankfurt, Germany. Frankfurt is one of the cities in the Cityscapes validation set, and its dense urban core provides many street-level viewpoints suitable for capture ([Figure 1](#)). We make no modifications to Arnis; the default block-to-OSM mapping is taken as given, and its limitations (e.g., narrow road strips relative to real-world road widths) propagate into our training data—a point that we return to in [section 5](#).

3.3. Flat-color resource pack and capture

The core of the pipeline is a Minecraft resource pack that replaces every block’s texture with a flat color that encodes its semantic class. We extract the visible block textures from the Minecraft client files and classify each block name by rule: blocks that Arnis uses for road surfaces are mapped to the road class; blocks whose names match vegetation-related terms are mapped to vegetation; non-physical or dynamic blocks are mapped to ignore; and the remaining blocks default to structure, since many blocks can be used for building. Each class is assigned a fixed RGB color—(255, 0, 0) for road, (0, 0, 255) for structure, and (0, 255, 0) for vegetation—and we write a 16×16 solid PNG with the assigned color for every block. Sky pixels are not handled by the pack; Minecraft renders the sky procedurally as a vertical gradient identified at post-processing ([Figure 2](#)).

To capture paired data, we walk the player to viewpoints throughout the world and at each one take an RGB screenshot, swap resource packs, take a second screenshot, then swap back. A Python script pairs timestamped screenshots into matched (`rgb`, `mask`) files. Graphics settings are held fixed to minimize lighting variation. We collected 300 paired screenshots in total.

3.4. Label post-processing

The raw masks are not yet usable as training labels for two reasons. First, in-game lighting introduces per-pixel color variation, so a pixel intended to be the structure’s blue color (0,0,255) is more often a darker variant of blue. Second, the pack does not render sky pixels at all and instead retains Minecraft’s procedural sky gradient, which spans from pale blue to white.

We resolve both with a per-pixel channel-dominance classifier: each pixel is assigned to one of the four classes based on which color channel dominates the others, with a separate rule for sky pixels (where all three channels are simultaneously bright). Pixels that do not match any of the rules are labeled as `ignore` and excluded from the loss. [Table 1](#) summarizes the resulting class distribution.

3.5. Training setup

We fine-tune SegFormer-B2 [17] (27.3M parameters), initialized from weights pretrained on ADE20K [18]. The 300 paired images are split 270/30. Training augmentations include random scaling, cropping to 512×512 , horizontal flipping, color jitter, random gamma correction, and Gaussian blur, with images normalized using ImageNet statistics. We intentionally use aggressive color jitter, serving as an implicit domain randomization in the spirit of Tobin *et al.* [14], encouraging the model to rely on shape and layout rather than the exact colors of Minecraft textures.

We train for 40 epochs with AdamW, using separate learning rates of $6e-6$ for the backbone and $6e-5$ for the four-class decode head, with linear decay, and a batch size of 8. The loss is per-pixel cross-entropy with `ignore_index` set to 255. We checkpoint at the best held-out Minecraft mIoU and use that checkpoint for all subsequent evaluation. Training runs on Google Colab’s NVIDIA A100 GPU and completes in approximately 30 minutes.

4. Experiments

4.1. Datasets and evaluation protocol

We evaluate the trained model on two test sets and compare it against one external baseline. The first test set is the

Class	Pixel fraction (%)
Road	20.7
Structure	34.2
Vegetation	33.7
Sky	11.2
Ignore	0.2

Table 1. Class pixel distribution in the 300-frame Frankfurt dataset after post-processing.



(a) Aerial view of the generated world.

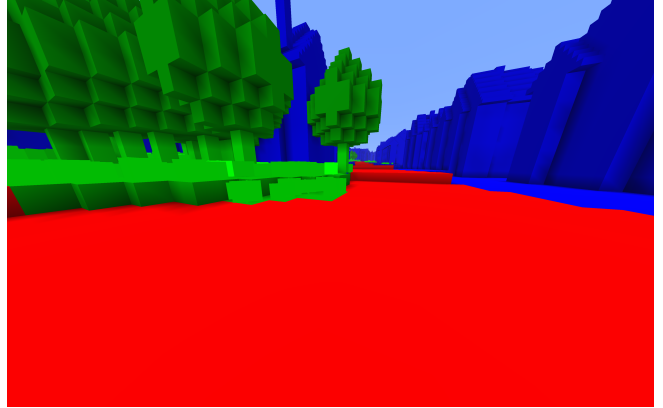


(b) Street-level capture viewpoint.

Figure 1. Minecraft world generated from OpenStreetMap data via Arnis, centered on Frankfurt. The block-level urban geometry preserved by Arnis is visible from above (a), with a representative ground-level capture viewpoint shown in (b).



(a) Default texture pack (RGB).



(b) Flat-color resource pack (raw mask).

Figure 2. Paired capture from a single viewpoint. Switching from the default texture pack (a) to our flat-color resource pack (b) replaces every block’s texture with the RGB color encoding its semantic class, while preserving viewpoint and lighting. Road appears red, structure blue, vegetation green; the sky retains its procedural gradient and is handled at post-processing.

held-out 30-image Minecraft validation split, used as an in-distribution sanity check. The second is the Cityscapes [3] validation set (500 images across Frankfurt, Lindau, and Münster), used to measure zero-shot transfer to real urban scenes. The baseline is the same SegFormer-B2 architecture, initialized from publicly released ADE20K-pretrained weights [18], and post hoc remapped to our four-class scheme; this measures how much of the Cityscapes performance can be obtained from existing scene-parsing pre-training alone, without using Minecraft data.

Cityscapes has 35 raw label IDs, which we remap directly to our four-class scheme: road maps to road; sidewalk, building, wall, and fence map to structure; vegetation maps to vegetation; sky maps to sky; and all remaining classes (vehicles, pedestrians, poles, traffic signs, terrain, parking, rail track, etc.) map to ignore.

For the ADE20K baseline, we use the public

nvidia/segformer-b2-finetuned-ade-512-512 checkpoint without modification. At evaluation time, we take the argmax over its 150 classes and apply a fixed lookup table that aggregates ADE classes by semantic correspondence to our scheme. Pixels where the baseline predicts an ADE class with no mapping to our scheme are excluded from the IoU computation rather than counted as misclassifications. This protocol is generous to the baseline, and the resulting numbers should be read as an upper bound on what scene-parsing pretraining alone can achieve in our four-class evaluation.

All images and masks are resized to 1024×512 , preserving Cityscapes’ 2:1 aspect ratio. We report per-class IoU and mean IoU (mIoU) over the four classes.

Method	road	structure	vegetation	sky	mIoU
Minecraft (in-distribution val)	0.91	0.87	0.93	0.94	0.91
Minecraft → Cityscapes	0.36	0.44	0.41	0.61	0.45
ADE20K → Cityscapes (baseline)	0.97	0.87	0.87	0.90	0.90

Table 2. Per-class IoU and mean IoU across evaluation settings.

4.2. Main results

Table 2 reports per-class IoU and mean IoU for the three evaluation conditions. The Minecraft-trained model achieves 0.91 mIoU on its in-distribution validation set, confirming that the pipeline produces labels learnable to high accuracy by a standard segmentation model. On Cityscapes, the same model achieves 0.45 mIoU, with substantial variation across classes: sky (0.61) and structure (0.44) are best, road (0.36) and vegetation (0.41) are weakest. The ADE20K-pretrained baseline, applied zero-shot, achieves 0.90 mIoU on Cityscapes—substantially higher than our Minecraft-trained model across all four classes

The main finding is that our Minecraft-trained model substantially underperforms the ADE20K-pretrained baseline despite having the same architecture. This gap does not arise from training failure: the model fits the Minecraft data well, and the 0.46 mIoU drop from in-distribution (0.91) to Cityscapes (0.45) is an empirical measurement of the sim-to-real gap at this fidelity level. Performance varies non-trivially across classes, suggesting that some semantic categories transfer better than others—a pattern that we examine next.

4.3. Analysis

4.3.1. Per-class behavior

The row-normalized confusion matrix on Cityscapes (Figure 3) reveals a strong asymmetry in the structure class: 96% of ground-truth structure pixels are predicted correctly, but the model also predicts structure for 63% of road pixels, 57% of vegetation pixels, and 34% of sky pixels. The result is a class with high pixel accuracy (0.96) but low IoU (0.44), indicating that many false positives are associated with this class.

We attribute this pattern to the absence of several Cityscapes categories from our Minecraft training data. Cityscapes contains vehicles, pedestrians, traffic lights, and riders, none of which our pipeline produces. When the model encounters pixels belonging to such objects (which we mark as ignored in evaluation but the model must still predict), it has no learned representation to draw on and defaults to the closest match in its training distribution—typically structure, which absorbs most vertical, built-looking surfaces. This fallback also contaminates nearby pixels at object boundaries, pushing predicted boundaries

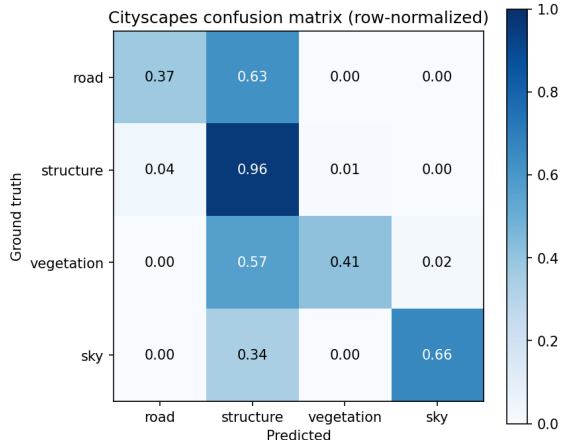


Figure 3. Row-normalized confusion matrix on the Cityscapes validation set. Each row represents a ground-truth class and sums to 1; the value at row i , column j is the fraction of class- i pixels predicted as class j . The structure column dominates: large off-diagonal mass in the road, vegetation, and sky rows reflects the model’s tendency to default to structure for content unseen during training.

outward and eroding the IoU of adjacent classes, most visibly road (see Figure 4).

Sky is the best-performing transferred class with an IoU of 0.61. This is unsurprising, as the sky is identified primarily by its image position and uniform color, both of which are well-preserved in Minecraft. In contrast, roads and vegetation exhibit distinct geometric mismatches. Roads in our training frames occupy a smaller fraction of the image than they do in Cityscapes because Arnis renders OpenStreetMap roads at a smaller width. As a result, the model has learned that roads are thin gray strips rather than the dominant ground surface in the lower image. Vegetation fails for a different reason: Minecraft trees are blocky, solid shapes, while Cityscapes trees are thin and branchy with see-through edges around the leaves—a shape that the training data does not capture.

4.3.2. Qualitative observation

Figure 4 shows representative Cityscapes predictions alongside the ground truth remapped to our four-class scheme. The fourth image illustrates the model’s best behavior: road and structure sky are recovered in approxi-

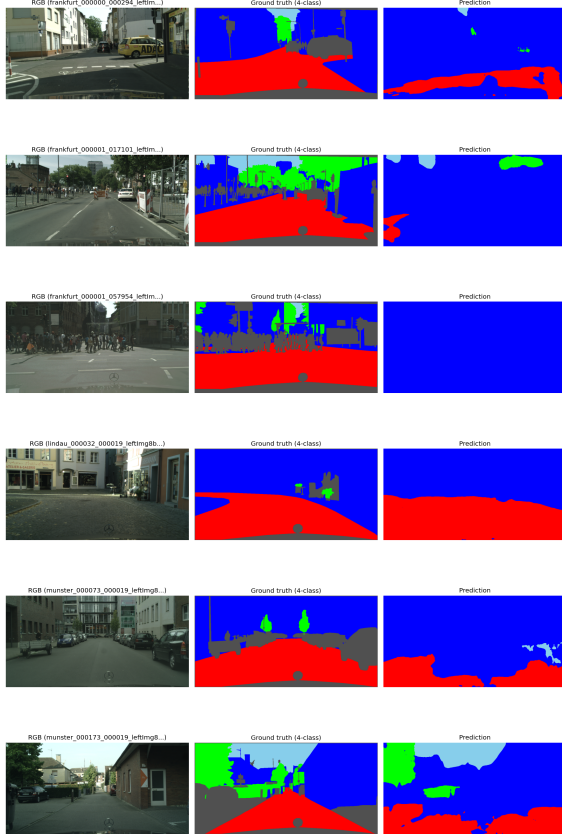


Figure 4. Qualitative predictions on Cityscapes validation images. Each row shows the input RGB image (left), the ground-truth labels remapped to our four-class scheme (middle), and the prediction from our Minecraft-trained model (right). Colors: road (red), structure (blue), vegetation (green), sky (light blue), ignore (gray). The model recovers coarse scene layout in clean street scenes but defaults to structure for vehicles, pedestrians, and other categories absent from the Minecraft training data, producing crescent-shaped intrusions into surrounding road and vegetation regions.

mately the correct regions. The remaining images expose the failure modes identified above. The fifth image, which shows parked vehicles, illustrates the structure-bias mode: the cars are predicted as structure, and this prediction bleeds into the surrounding road region. The third image, dominated by a crowd of pedestrians, is the most extreme case—the model collapses to predicting structure across the entire frame. In all examples, the failures correspond to the mismatches between our training distribution and Cityscapes, as analyzed above.

4.3.3. In-distribution check

For completeness, Figure 5 shows predictions on held-out Minecraft frames. They closely match the ground-truth labels, with smooth boundaries and no visible mode collapse, ruling out training failure as an explanation for the

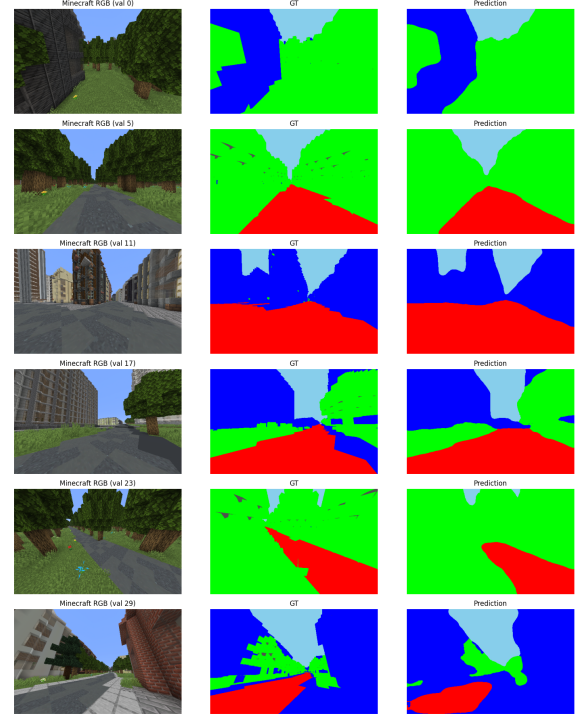


Figure 5. Qualitative predictions on held-out Minecraft validation images. Each row shows the input RGB image (left), ground-truth labels (middle), and our Minecraft-trained model’s prediction (right); colors are as in Figure 4. Predictions match the ground truth closely across all examples, with smooth class boundaries and no visible mode collapse. This rules out training failure as an explanation for the Cityscapes gap and confirms that the gap arises from the domain shift rather than from the training procedure.

Cityscapes gap and confirming that the drop arises from domain shift rather than the training procedure.

5. Discussion

5.1. The pipeline-baseline gap

The 0.45 mIoU shortfall against the 0.90 ADE20K-pretrained baseline is the paper’s primary negative result. We do not attempt to rationalize it; the baseline carries semantic knowledge from a real-image dataset (ADE20K) containing urban scenes, and despite our model being initialized from the same weights, fine-tuning on Minecraft labels appears to overwrite that knowledge with a narrower distribution shaped by the synthetic source.

A natural follow-up is to ask whether a different fine-tuning procedure (e.g., freezing the backbone or using a much smaller learning rate) would preserve more of the baseline’s real-image knowledge while still benefiting from Minecraft supervision. We attempted a brief hybrid experiment of this kind, but the results were inconclusive within the time available. We leave a careful study of fine-tuning

regimes that combine real-image and synthetic supervision to future work.

5.2. Implications for low-fidelity synthetic data

The broader question of whether fidelity matters and by how much admits a partial answer. Even with the advantage of ADE20K-pretrained initialization, a SegFormer model fine-tuned on Minecraft does not achieve the performance of the same model evaluated zero-shot from its ADE20K weights. Thus, fidelity matters substantially. However, it matters non-uniformly across classes. Categories whose visual signature survives extreme stylization (sky, dominant structures) transfer measurably even from a source as low-fidelity as Minecraft, while texture-dependent categories and those without analogs in the source do not. The value of a synthetic source, therefore, depends not only on average fidelity but on which axes it preserves: shape, layout, color statistics, object inventory, and geometric resolution.

A direction suggested by our results, other than simply increasing texture fidelity, is to expand the synthetic source’s object inventory. Adding even crude vehicle, pedestrian, and pole categories could potentially address the dominant failure mode we observe. Whether this is best done through Minecraft itself or through a combination of synthetic and real data is an open question worth investigating.

6. Limitations and Future Work

6.1. Limitations

Several limitations qualify the conclusions above.

Scale: We collected only 300 paired images, all from a single city; a larger-scale collection across cities, lighting, and viewpoints would likely improve absolute performance. However, based on the failure modes identified in Section 4, this alone would not close the gap to the ADE20K baseline.

Capture protocol: Screenshots were captured manually, which limits the diversity and total volume of training data.

Class scheme: The four classes were chosen for unambiguous mapping from Minecraft blocks rather than downstream utility; a more practical scheme would include vehicles, pedestrians, and other categories our pipeline cannot currently produce.

No domain adaptation: We evaluate the unadapted regime to isolate what transfers from Minecraft alone, but a more thorough evaluation of the data’s utility would combine our training procedure with standard adaptation methods.

6.2. Future work

Several directions build directly on our pipeline.

Automated traversal: A scripted in-game agent could capture viewpoints systematically across a city (vehicle-height camera, fixed FOV, regular spacing), producing thousands of frames without manual labor.

Combination with real data: The most likely path to a positive result lies in combining Minecraft-generated data with real-image supervision, either through joint training or through more carefully tuned fine-tuning regimes than the one we attempted.

Auxiliary signal for Minecraft-based RL agents: Existing Minecraft RL agents [1, 6, 9, 10] perceive their environment through raw RGB images; equipping them with semantic segmentation produced for free by our pipeline at training time could enable forms of structured perception that would otherwise require manually engineered features.

Pipeline extensions: Smaller extensions to the pipeline itself include customizing Arnis’s block selections to ensure distinct classes use disjoint block sets, and randomizing block textures across captures for Tobin-style domain randomization at no additional cost.

7. Conclusion

We presented a pipeline for generating semantic segmentation training data from procedural Minecraft worlds, combining OpenStreetMap-derived geometry, a flat-color resource pack for label extraction, and lightweight post-processing. We used this pipeline to investigate whether extremely low-fidelity synthetic data can train models that transfer to real urban scenes. The resulting model achieves 0.91 mIoU in-distribution but only 0.45 on Cityscapes, substantially below an ADE20K-pretrained baseline of 0.90. Class-wise analysis reveals that large position-defined regions (sky, dominant structures) transfer measurably, while categories absent from the synthetic source (vehicles, pedestrians, urban furniture) account for most of the gap through default-class substitution. The pipeline—our paper’s primary contribution—provides a low-cost, fully automated tool for further investigation into how fidelity, scale, and scene composition affect synthetic-to-real transfer.

Ethics

This work uses Minecraft (commercial software), OpenStreetMap (open-licensed geographic data), and the Cityscapes dataset (academic research license). The synthetic data we generated contains no representations of people, vehicles, or other personally identifying content; this is a privacy advantage of synthetic urban data over real-image alternatives.

Two potential concerns are worth noting. First, our synthetic data is geographically narrow: the Frankfurt-derived OpenStreetMap source represents a specific European urban style, and a model trained on this data should not be expected to generalize to non-European cities or to urban

styles underrepresented in OpenStreetMap without further data collection. Second, semantic segmentation of urban scenes has dual-use applications, including autonomous vehicles and surveillance. While these concerns are not specific to our pipeline, we acknowledge them as inherent to the broader research of automated urban scene understanding

Acknowledgments

I used Claude to brainstorm ideas for my project and draft the code for the experiments.

References

- [1] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampeiro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos, 2022. [2](#), [7](#)
- [2] Gabriel J. Brostow, Julien Fauqueur, and Roberto Cipolla. Semantic object classes in video: A high-definition ground truth database. *Pattern Recognition Letters*, 30:88–97, 2009. [1](#)
- [3] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding, 2016. [1](#), [2](#), [4](#)
- [4] Aysegul Dundar, Ming-Yu Liu, Ting-Chun Wang, John Zedlewski, and Jan Kautz. Domain stylization: A strong, simple baseline for synthetic to real image domain adaptation, 2018. [2](#)
- [5] Louis Erbkamm and Arnis Contributors. Arnis: Generate any place from the real world in Minecraft. <https://arnismc.com/>, 2024. Open-source software, accessed 2025. [2](#), [3](#)
- [6] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge, 2022. [2](#), [7](#)
- [7] Adrien Gaidon, Qiao Wang, Yohann Cabon, and Eleonora Vig. Virtual worlds as proxy for multi-object tracking analysis, 2016. [2](#)
- [8] Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. Imagenet-trained cnns are biased towards texture; increasing shape bias improves accuracy and robustness, 2022. [2](#)
- [9] William H. Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso, and Ruslan Salakhutdinov. Miner!: A large-scale dataset of minecraft demonstrations, 2019. [2](#), [7](#)
- [10] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024. [2](#), [7](#)
- [11] OpenStreetMap contributors. OpenStreetMap. <https://www.openstreetmap.org>, 2026. Data available under the Open Database License. [2](#)
- [12] Stephan R. Richter, Vibhav Vineet, Stefan Roth, and Vladlen Koltun. Playing for data: Ground truth from computer games, 2016. [1](#)
- [13] German Ros, Laura Sellart, Joanna Materzynska, David Vazquez, and Antonio M. Lopez. The synthia dataset: A large collection of synthetic images for semantic segmentation of urban scenes. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3234–3243, 2016. [2](#)
- [14] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017. [2](#), [3](#)
- [15] Jonathan Tremblay, Aayush Prakash, David Acuna, Mark Brophy, Varun Jampani, Cem Anil, Thang To, Eric Cameracci, Shaad Bochooon, and Stan Birchfield. Training deep networks with synthetic data: Bridging the reality gap by domain randomization, 2018. [2](#)
- [16] Yi-Hsuan Tsai, Wei-Chih Hung, Samuel Schulter, Kihyuk Sohn, Ming-Hsuan Yang, and Manmohan Chandraker. Learning to adapt structured output space for semantic segmentation, 2020. [2](#)
- [17] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers, 2021. [2](#), [3](#)
- [18] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene Parsing through ADE20K Dataset. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5122–5130, 2017. [2](#), [3](#), [4](#)